

SESSION 2 — PRACTICAL WORKFLOWS

Practical AI for Academics

Session 2 — Practical workflows

Claes Bäckman

Leibniz Institute for Financial Research SAFE

What do you actually need from AI?



Sophia Kazinnik  @SophiaKazinnik · 13h



I got *a lot* of questions on sophisticated workflows, agents, skills, orchestration, and increasingly elaborate setups (e.g., in the surveys I ran, the most requested topic by far was agents: "agent work," "agent building," "I want to learn how to work with AI agents.").

What do you actually need from AI?



Sophia Kazinnik ✓ @SophiaKazinnik · 13h



I got *a lot* of questions on sophisticated workflows, agents, skills, orchestration, and increasingly elaborate setups (e.g., in the surveys I ran, the most requested topic by far was agents: "agent work," "agent building," "I want to learn how to work with AI agents.").

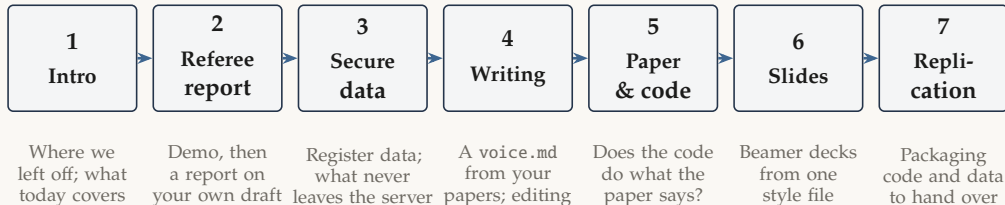


Sophia Kazinnik ✓ @SophiaKazinnik · 13h



Lets be honest here. Most of our empirical research tasks are still fairly bounded: e.g., scrape data, clean data, run analysis, classify text, summarize documents, produce a forecast, repeat. You usually do not need a persistent autonomous agent, elaborate memory architecture, etc., to do those tasks well. In many cases, in fact, this is unproductive and adds complexity, cost, new failure points..

Session 2 agenda



Slides, skills, and guides for both days: claesbackman.com/ai_workshop

SECTION 1

A referee report on your paper

*Pick one draft you would like another pair of eyes on.
Preferably a relatively close-to-finished draft.*

Then write /review-paper

/review-paper on a real draft, eight subagents in parallel.

Setup

- One of my own papers, in its own folder
- .tex source, .bib, supporting materials
- One command: /review-paper with a target journal

Output

- A structured markdown report
- Referee-style comments, grouped by theme
- Specific line and section references

Eight subagents running in parallel

- Spelling, grammar & style
- Internal consistency & cross-references
- Unsupported claims & identification
- Mathematics, equations & notation
- Tables, figures & documentation
- Referee assessment: positioning & fit
- Contribution advocate
- Contribution skeptic

Pattern: eight narrow critics beat one diffuse one.

Run a referee report on your own writing.

1. Open one of your own drafts in VS Code
2. Make sure it is in markdown or LaTeX — convert if needed
3. Run `/review-paper-light` for a 1–2 minute version, or `/review-paper` for the full eight-agent run
4. Read the output. It's not all correct!

Debrief questions

- What did the report catch that you would not have seen by yourself?
- What did it get wrong?
- Which subagent gave you the most useful feedback?

A skill is a markdown file the agent loads on demand — invoke with /name.

A skill lives at `.claude/skills/your-skill/SKILL.md` (or `~/ .claude/skills/` for every project):

```
---  
name: review-paper  
description: Run an 8-agent pre-submission referee report for an academic paper  
targeting a specified journal  
---  
You are coordinating a rigorous pre-submission review of an  
academic economics paper. You will run 8 specialized review agents  
in parallel and consolidate their findings into a structured report.
```

Invoke with `/review-paper`. Claude loads the instructions and runs them.

CLAUDE.md is always loaded, skills are on demand.

CLAUDE.md — always loaded

- Loaded into context every session
- Background, conventions, voice
- “How I want you to work with me”

Cost: permanent context budget.

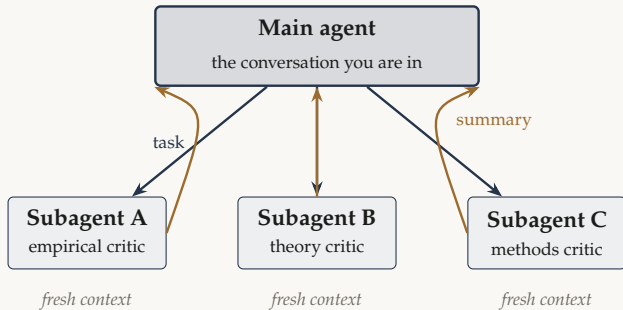
Skill — on demand

- Loaded only when invoked with /name
- A specific workflow, with structured output
- “Do this thing for me”

Cost: zero until triggered.

KEY TAKEAWAY. *If you find yourself pasting the same prompt multiple times, you can ask Claude to make it into a skill.*

Subagents add two things the main agent cannot do alone — parallelism and context isolation.



KEY TAKEAWAY. *The /review-paper skill spawns **eight** subagents.*

*Now launch another agent to critique the referee report
you just generated — ask Claude to write the
instructions.*

The second-opinion pattern — spawn an adversary to check on the work.

The main agent sees its own prior answer and has reason to defend it.

Why a fresh subagent is different

- No chat history: no anchoring on what you already wrote
- No conversational momentum: no reason to agree with the main agent
- Clean context: can hold the whole artifact at once

When to spawn one

- Before accepting a long answer: *“is this actually right?”*
- After a long refactor: *“what did we break?”*
- When you suspect flattery: *“give me the honest version”*
- Before sending: *“read this as a hostile reader”*
- After doing automatic edits: *“look over the changes we did”*

KEY TAKEAWAY. The phrases **“spawn a subagent”** or **“launch an agent”** are the trigger words. The main agent knows what to do and will write the instructions for you.

What the demo is meant to show

1. **Grounded output.** The report is about *your* draft, not a generic checklist
2. **Subagents are why this works.** Eight narrow critics catch things one diffuse reader misses
3. **Not a replacement for a referee.** It is a pre-flight check before you submit
4. **Transparent.** The skill is a ~200-line markdown file — fork it, modify it, make it yours

KEY TAKEAWAY. *The point is not to trust the report. The point is to make hard feedback cheap enough that you ask for it before submitting — not after the desk reject.*

Other tools generate referee reports too

Tool	What it does	Pricing
Refine.ink	Referee-style report on a paper PDF	<i>Paid</i>
coarse.ink	Lightweight version of the same idea	<i>≈ free</i>
Referee 2	Paid referee-report tool	<i>Paid</i>
Stanford Agentic Reviewer	Open agentic reviewer	<i>Free</i>

KEY TAKEAWAY. *The advantage of running your own skill is that it lives next to your draft, knows your conventions, and you can edit what it checks.*

SECTION 2

Working with secure data

You should never paste any sensitive data or results into Claude

Files or chats that Claude read go to Anthropic servers

Confidentiality and data

- **Never paste** confidential microdata, register data, or unpublished co-authored work.
- **“Training on your data.”** Some consumer tiers may use your chats by default. Know the setting; prefer business/team tiers for anything sensitive.
- **When in doubt, abstract it.** Ask about a made-up example or use a synthetic dataset.
- **Check institutional policy** before putting any work material into any AI tool.

Secure servers split the workflow

What Claude can still help with on your laptop

- Your .do files
- Writing and iterating on empirical strategy

Claude cannot run the files or read the logs on the server, but can still be very helpful in a number of ways

Two artifacts to keep local

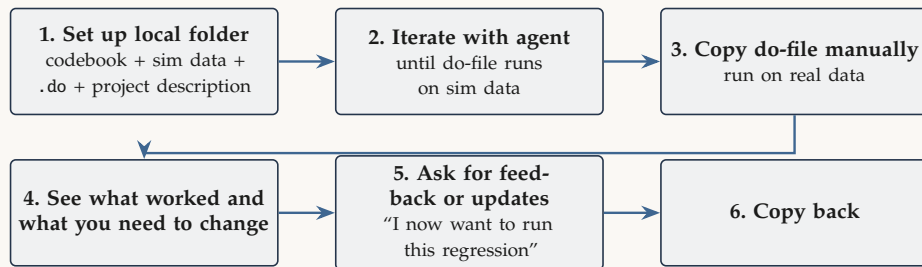
1 Codebook in markdown

- Convert once (/pdf-to-markdown)
- Variable names, labels, value ranges, missing-value codes
- Agent reads it before writing recode, label define, merge

2 Simulated dataset

- Same variable names, types, ranges — not real values
- AER/JF/ReStud replication packages often ship with these
- Or generate from the codebook

A workflow for secure data on servers



KEY TAKEAWAY. *The agent never sees the real data (and you should never paste any result from the server into Claude)—but it sees everything that explains the data. That is usually enough to write correct Stata code.*

What synthetic data does and does not buy you

Tests that the code RUNS

Variable names, merge keys, reshapes, type handling, sign conventions, the shape of the panel. All the things that produce an error message.

Does NOT test that it is RIGHT

Correlations, magnitudes, selection, missingness patterns. The fake data can only contain what someone thought to put in it

SECTION 3

Writing with agentic AI

Use AI to edit your draft, don't edit the AI's draft

Writing is thinking

If you remove too much of the writing friction, it will be harder to figure out what your paper is about

People are (rightfully) annoyed with AI writing

It is now easy to spot AI writing, especially with tools like Pangram. You don't want people to discount your idea

AI detectors can tell AI generated writing

The most popular current detector is **Pangram**, with very low reported false-positive rates.

How they work. A classifier trained on the fingerprint of model text.

Does it work? It seems to, and it likely will in the future

A `voice.md` file with your style

When editing my prose, match the voice in `voice.md`.
Do not use em-dashes, bullet lists, or passive constructions unless the original does.
Do not use "it's not x, it's y"-style writing.

What goes in it

- 5–10k words of your prose — intros, abstracts, paragraphs you are proud of

Hands-on (15 min) — build a `voice.md` from your own writing.

We use a skill written by Mihail Velikov: `voice-extractor`.

1. Upload 4 PDFs of your own writing to a folder
2. Call the skill: `/voice-extractor sample-papers`
3. Read through `voice.md` and verify — edit anything that does not sound like you

What goes in. Intros, abstracts, paragraphs you are proud of. 5–10k words is plenty.

/prose-revision fixes the order of the ideas before the choice of words.

What comes back. A diagnosis of the idea flow first, then two or three rewrites that differ in how far they restructure, and a short note on the tradeoff between them. Claims, citations, and notation come through unchanged.

Where it stops. It will not add or drop an idea without saying so, and it is not a proofreader. Revision comes first, the typo pass comes last.

From https://x.com/alz_zyd_

SECTION 4

Verifying changes

A skill: `/explain-diff`

`/explain-diff` checks what changed within a Github repo and generates a html site that you can open

- **Verification.** It diffs generated tables, compares N and headline coefficients across logs, and quotes numbers only from output files it has opened.
- **Output.** One offline HTML page in five sections: what the code did before, what changed and why, consequences for the results, a code walkthrough grouped by purpose, and a five-question quiz. Saved outside the repo with a plain-text summary in the chat.

Dropbox or Github?

	GitHub	Dropbox
Sync	Manual push and pull	Automatic, within seconds
History	Full git history, branches	Dropbox revisions only
Best for	Solo work, versioned drafts	Live coauthoring in Overleaf
Conflicts	Git merge	“Conflicted copy” files

Two routes from Overleaf to your computer

Setup A GitHub via GitHub Desktop

1. Overleaf: Menu → Integrations → GitHub → **Export Project to GitHub**
2. GitHub Desktop: File → Clone repository, pick the repo and a local folder
3. Open that folder in VS Code
4. Edit in VS Code. In GitHub Desktop: one-line summary, **Commit to main, Push origin**
5. Overleaf: Menu → GitHub → **Pull GitHub changes into Overleaf**
6. Coauthor edited in Overleaf? **Push Overleaf changes to GitHub**, then pull in GitHub Desktop

Accept the TeX `.gitignore` template: `no .pdf` or `.aux` in the history.

Setup B Dropbox

1. Install Dropbox and let the initial sync finish
2. Overleaf: Menu → Dropbox Sync → **Link**. Overleaf creates `Apps/Overleaf/` with one subfolder per project
3. Open the project folder in VS Code and edit as normal — every save reaches Overleaf within seconds, and Overleaf edits flow back
4. Compile in Overleaf as usual, or locally with `latexmk -pdf`

No commit step. Good for active coauthoring, less good if you want a paper trail.

SECTION 5

Slides

Making slides with AI

The old path

- Open Beamer
- Write LaTeX by hand
- Fight the syntax
- Iterate slowly

Hours of mechanical work between idea and first draft.

The Claude Code path

- Open the project folder
- Hand Claude a file: “*Make a Beamer deck based on @paper.tex*”
- Compile, look at the output, iterate

This deck was built that way.

KEY TAKEAWAY. *This also means that we have to practice presentations in a different way*

Including a style file gives Claude guidance to match the look you want.

Without a style file. Claude defaults to generic Beamer themes.

With a style file. The deck inherits your colour palette, frame structure, and title conventions.

How to build a `slides-style.md`

- Ask Claude to analyze your old presentations or to make up something that looks good
- Save the result as `slides-style.md` in the project folder
- Reference it whenever you ask for a new deck

AI should let us do new things, not only old things faster

Borrow from people who explain things for a living

Nicky Case's explorable explanations, Bret Victor, Steven Strogatz: a clear narrative, interactive elements that carry the message, a clean visual language.

Examples

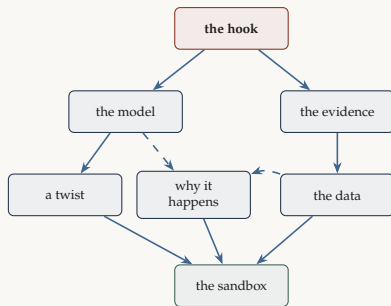
Examples: a [housing-returns paper](#) and a [paper on financial advice from LLMs](#). The skill: github.com/claesbackman/AI-research-feedback, [Skills/explorable-deck](#). Adapted from Nityesh Agarwal's skill for explorable websites.

Other fields have worked out how to explain their work

/explorable-explanation builds a website as a non-linear tree of short HTML decks, one idea per slide, with forks placed where the reader becomes curious.

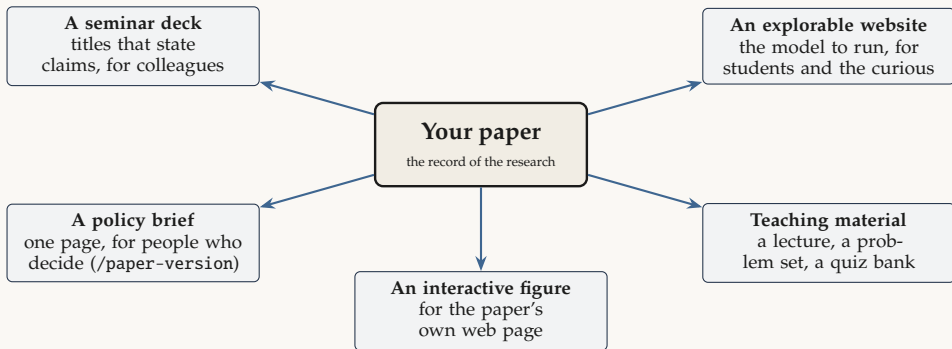
How it works

1. Reads the source material, then writes `plan.md` like a script: the hook, what the reader sees and does, and where the branches go
2. Shows you the first batch before it builds the rest
3. Screenshots every slide and reworks the ones that break the craft rules
4. Sends adversarial subagents to judge writing, interface, visuals and teaching
5. A “creative director” subagent gives it a visual identity; an impatient-reader subagent flags every slide where the flow breaks



each box is a short deck; dashed arrows are forks the reader chooses from inside a slide

We do not have to be bound by the PDF



The paper is the source of the claims. But AI makes it feasible to communicate the claims in a different ways to different audiences.

SECTION 6

Teaching in times of AI, continued

Using AI to teach more

Teach more with AI

4. **Help students learn more efficiently.** Class-specific tutoring, spaced repetition, mastery learning at scale.
5. **Use AI to improve your own teaching.** Find out where you were confusing before the exam tells you.
6. **Personalise assignments.** Problems at the level each student is struggling with.
7. **Raise standards.** If research and drafting are cheaper, the bar moves up.

Implementing tutors, misconception digests, levelled problem sets.

Course tutor

A shared assistant (custom GPT, Claude project, NotebookLM) loaded with *your* lecture notes and problem sets.

System prompt essentials: use the course's notation, never give the answer, ask the student to explain first, return to topics they missed.

Misconception digest

Each week, export the questions students asked (tutor logs, forum, email), anonymised.

Prompt: “Group these by underlying misconception, rank by frequency, and point to the lecture slide most likely to have caused each.”

Levelled problem sets

One problem, three difficulty tiers. Assign the tier by last week's quiz result.

Prompt: “Write this problem at three levels: scaffolded, standard, extension. Same concept, full solutions, and a rubric I can grade against.”

KEY TAKEAWAY. *Student questions are personal data. Anonymise before exporting and check the institutional policy once.*

SECTION 7

Conclusions

Key message: Context improves AI tools tremendously, and the goal is to provide the right context

Key difference: how you provide the useful context and how easy it is

Q & A

Open floor. Email: claes.backman@gmail.com |
Slides, skills, guides: claesbackman.com/ai_workshop

SECTION 8

Setting up your workspace

Five steps get VS Code + Claude Code working in under five minutes.

1. Open VS Code and the Extensions panel (Ctrl+Shift+X / Cmd+Shift+X)
2. Search “**Claude Code**” and install *Claude Code for VS Code* (extension ID `anthropic.claude-code`)
3. Restart VS Code — Claude Code opens in the sidebar
4. Sign in with your Claude account (Pro/Max/Team/Enterprise all include Claude Code) or an API key
5. Open your **full project folder** (File > Open Folder) — never a single file

What an IDE actually is. Editor + file explorer + terminal + extensions, in one window. VS Code is the dominant free one. Think of it as Word or R Studio for code, LaTeX, markdown, and plain text.

How to get started

Get the skills from claesbackman.com/ai_workshop — download the zip, or paste the one-line terminal command on that page. Source and updates: github.com/claesbackman/AI-research-feedback.

Unzip, then copy each skill folder (review-paper, review-paper-light, ...) into `~/.claude/skills/` — a hidden folder under your user folder (Windows: `C:\Users\you\.claude\skills`)

- To show hidden folders: Open File Explorer, click the View tab at the top, select Show, and check Hidden items
- A skill in `~/.claude/skills/` works in every project; one in `project/.claude/skills/` only in that project

Restart VS Code

The skill loads when you type `/skill-name`

A handful of extensions cover Stata, Python, LaTeX, and data

Stack	Extension ID	What it does
Stata	tmonk.stata-workbench	Run Stata from the editor
Python	ms-python.python	Syntax, IntelliSense, run button
LaTeX	james-yu.latex-workshop	Compile on save, side-by-side PDF preview, inline errors
Data	mechatroner.rainbow-csv	Colour-coded CSV columns; SQL-like filtering
	tomoki1207.pdf	Renders PDFs inline — read codebooks next to your code

KEY TAKEAWAY. *Install Python even if you do not write it. The agent uses Python as glue for many different tasks.*

You can link Overleaf to your own computer and work in that folder

Why bother? Overleaf has no Claude Code, no Codex, no Copilot, no proper diff view, no terminal. The fix: write in VS Code and let Overleaf follow what you produce.

Requirement. Both routes need **Overleaf Premium** — many universities provide it through the institutional login.

Every Claude Code session: open the folder, set context, then prompt.

1. Open VS Code and the project folder (File → Open Folder)
2. Open the Claude Code panel in the sidebar — or the terminal (Ctrl+`) and type `claude`
3. Confirm Claude is pointed at the **full project folder** — not a single file
4. Send a first orienting prompt

First prompt to try:

“Read the files in this folder and tell me what this project is about.”

Expected behaviour. Claude reads the folder, summarises the structure, and asks you what you want to do next.

CLAUDE.md is the standing-instructions file that is always loaded

```
# About me
```

```
I am a researcher in [field]. I write papers, lecture notes, policy pieces.
```

```
# How I want you to work with me
```

- Ask clarifying questions before generating long output.
- Keep a skeptical, critical tone in feedback. Do not flatter.
- When editing, preserve my voice. Avoid generic LLM phrasing.
- Avoid bullet lists and passive voice in formal writing.
- Cite the specific line or section you are commenting on.

```
# Things to avoid
```

- Do not fabricate citations.
- Do not over-claim causality when summarising empirical work.
- Do not insert emoji or markdown decorations in formal documents.

A few files hold your durable project context

File	Use it for
<code>project/CLAUDE.md</code>	Project instructions. Dataset structure, sample restrictions, paper title, coauthor conventions, which script produces which output.
<code>~/.claude/CLAUDE.md</code>	Personal preferences across every project: writing style, default tone, citation conventions. Project instructions are loaded later and usually carry more weight.
<code>voice.md</code>	Writing samples. Paragraphs, abstracts, and tone notes that Claude should match when editing prose.
<code>.claude/skills/...</code>	Reusable workflows. Named tasks you want to run again without rewriting the prompt.

KEY TAKEAWAY. *Keep secrets and raw confidential data **out of all of them** — these files are read into context on every session and may be sent to the model provider.*

Bootstrap CLAUDE.md in two minutes — type `/init`.

The built-in command: type `/init` in the Claude Code panel. Or spell it out:

“Read this project folder and draft a CLAUDE.md file with the project structure, conventions, and instructions you should follow. Do not edit anything else yet.”

What Claude does

- Reads the folder
- Infers languages, structure, conventions
- Generates a CLAUDE.md draft

Then you edit it. Add tone preferences, things to avoid, sample restrictions Claude could not infer.

Permissions follow a tiered model

Plan mode

Reads and proposes; cannot write.

Manual

Asks before each edit or command.

Edit automatically

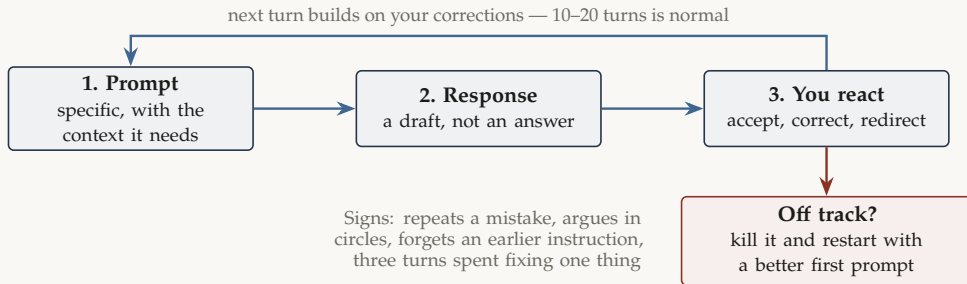
Claude will edit your selected text or the whole file

Auto mode (default mode now)

Claude will approve actions that pass a safety check and pause for anything risky

Claude Code should stay inside the folder you opened, but this is not certain. Use `/permissions` to see the current rules, and allow- or block-list specific commands (`Rscript`, `pdflatex`, `rm`) in `settings.json`.

A session is a conversation, not a query



Continue the conversation. The context carries your corrections, so later answers improve: *“Shorter, and use the Danish case.” “You misread Table 2 — look again.” “Now the same for Section 3.”*

Kill it when you veer off track. Start fresh and put what you learned into the first prompt — or into `CLAUDE.md` so you never repeat it.

Appendix — CLAUDE.md starter template.

About me

I am a researcher in [field].

How I want you to work with me

- Ask clarifying questions before generating long output.
- Critical, skeptical tone in feedback. Do not flatter.
- When editing, preserve my voice (see voice.md). No generic LLM phrasing.
- Avoid bullet lists and passive voice in formal writing.
- Cite the specific line or section you are commenting on.

Things to avoid

- Do not fabricate citations.
- Do not over-claim causality.
- Do not insert emoji or markdown decorations in formal docs.

Appendix — instructor's skill library.

Skill	Use
<code>/review-paper</code>	Pre-submission referee report, 8 agents
<code>/review-paper-light</code>	Fast 2-agent version, $\approx$ 1 minute
<code>/review-paper-code</code>	Paper + code alignment review
<code>/review-grant</code>	Pre-submission grant review, 6 agents
<code>/review-pap</code>	Pre-analysis-plan review
<code>/audit-analysis</code>	Adversarial audit of changed analysis code
<code>/explain-diff</code>	Explain a code change as an HTML page with a quiz
<code>/paper-version</code>	Paper $\rightarrow$ policy brief / 1-page / 5-page
<code>/pdf-to-markdown</code>	Convert PDFs to readable markdown

All at github.com/claesbackman/ai-research-feedback (MIT), linked from claesbackman.com/ai_workshop. Fork them, modify them, make them yours.

Appendix — file format quick reference.

Format	Handling
.md, .tex, .txt, .bib	Native — no conversion
.py, .R, .do, .ipynb	Native — read, edit, run
.csv	Native up to a few thousand rows
.docx	Convert: <code>pandoc file.docx -o file.md</code>
.xlsx	Export to CSV first
.dta, .rds	Script-based conversion to CSV
PDF (born-digital, short)	Native; expect table mangling
PDF (long, scanned, equations)	Convert to markdown first

Appendix — signs of session drift.

- Claude ignores `CLAUDE.md`
- Repeats mistakes you already corrected
- Invents files that do not exist
- Gives circular answers
- Forgets the last three turns

Fix. Start a fresh session or close and reopen. Re-state the task. Re-attach the relevant file.

30+ turns or the task has shifted → start fresh. Always.

/review-paper-code asks whether the code does what the paper says.

What it does, in four steps

1. Finds your main `.tex` and your code folder on its own — Stata, R, Python
2. Reads the paper for the sample, the specification, and the headline numbers
3. Runs two agents at once: reproducibility, and paper-to-code mapping
4. Reconciles the two and writes `reviews/code_review_report.md`

Call it: `/review-paper-code`, adding `full` to review every file it finds.

What comes back

- **Reproducibility checklist:** paths, seeds, ungenerated outputs, dependencies, run order
- **Paper-code map:** matched, worth verifying, or not found in the files reviewed
- **Likely discrepancies:** where the two disagree
- **Next steps,** ordered by importance

SECTION 9

A replication package

A replication package is a README, a master script, and portable paths.

What has to be in the folder

1. A README: data sources, access terms, software versions, runtime
2. A table mapping each exhibit to the script that makes it and the file it writes
3. One master script that runs the pipeline in order, relative paths, seeds set
4. The data you may ship, and access instructions for the data you may not

What to ask the agent

- “Read Code/ and draft the README, including the exhibit-to-script table”
- “List every input file the code reads; flag the ones not in this folder”
- “Make every path relative and write `master.do`”

Then verify: run it in a fresh folder and diff the tables against the paper.

KEY TAKEAWAY. *With register data the code and the README are the package — the data never leaves the server.*
